Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ» (ТУСУР)

Кафедра автоматизированных систем управления (АСУ)

Создание веб приложения Flask

Отчет по лабораторной работе №3 по дисциплине

«Разработка веб-сервисов для научных и прикладных задач»

Студент гр. 431-3

_____Андреев.Д.П.

«____» _____2022 г.

Руководитель

Доц. каф. АСУ,

канд. техн. наук.

_____ Суханов.А.Я.

«____» _____ 2022 г.

Томск 2022

Содержание

1 Реализация web-приложения для классификации изображений 2
1.1 Содержимое и структура каталогов 2
1.2 Содержимое файлов 2
Содержимое файла .travis.yml 2
Содержимое файла Procfile 2
Содержимое файла requirements.txt 2
Содержимое файла runtime.txt 3
Содержимое файла client.py 3
Содержимое файла net.py 4
Содержимое файла st.sh 5
Содержимое файла wsgi.py 5
Содержимое файла Net.html 5
Содержимое файла simple.html 6
Содержимое файла file.xml 7
1.3 Результат работы 8
2 Задание по варианту 10
Содержимое файла start.py: 10
Содержимое файла net.html: 11
Результат работы приложения: 13
3 Вывод 16

1 Реализация web-приложения для классификации изображений

1.1 Содержимое и структура каталогов

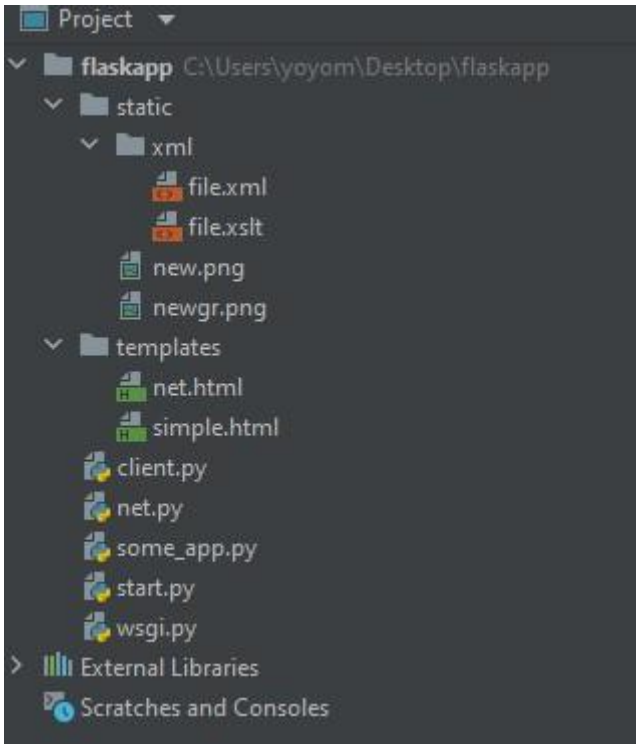

Рис. 1.1 – содержимое основной папки и папки flaskapp

1.2 Содержимое файлов

Содержимое файла .travis.yml

```
language: python python: -
"3.7.6" before_install: - chmod
+x ./flaskapp/st.sh install:
- pip install -r requirements.txt
script: - cd flaskapp - ./st.sh
```

Содержимое файла Procfile

```
web: gunicorn wsgi:app -b 0.0.0.0:$PORT --chdir flaskapp
```

Содержимое файла requirements.txt

```
gunicorn==20.1.0 Flask==2.0.1
requests==2.25.1 Flask-
Bootstrap==3.3.7.1
Flask-WTF==0.15.1
Pillow==8.2.0
```

```
tensorflow==2.4.0
Keras==2.4.3
lxml==4.6.3
```

Содержимое файла runtime.txt

```
python-3.9.5
```

Содержимое файла client.py

```
import requests

r = requests.get('http://localhost:5000/')
print(r.status_code)
print(r.text)
r = requests.get('http://localhost:5000/data_to')
print(r.status_code)
print(r.text)
import os
from io import BytesIO
import base64
img_data = None
# создаем путь к файлу
path = os.path.join('./static','new.png')
# читаем файл и ренкодируем его в строку base64
with open(path, 'rb') as fh:
    img_data = fh.read()
    b64 = base64.b64encode(img_data)
    jsondata = {'imagebin':b64.decode('utf-8')}
res = requests.post('http://localhost:5000/apinet', json=jsondata)
if res.ok:
    print(res.json())
try:
    r = requests.get('http://localhost:5000/apixml')
    print(r.status_code)
    if(r.status_code!=200):
        exit(1)
    print(r.text)
except:
    exit(1)
```

Содержимое файла net.py

```
import random
import keras
from keras.layers import Input
from keras.models import Model
from keras.applications import MobileNetV2
from keras.applications.mobilenet_v2 import preprocess_input, decode_predictions



import os
# модуль работы с изображениями
from PIL import Image
import numpy as np
# для конфигурации gpu
from tensorflow.compat.v1 import ConfigProto
from tensorflow.compat.v1 import InteractiveSession
# настраиваем работу с GPU
config = ConfigProto()
config.gpu_options.per_process_gpu_memory_fraction = 0.7
config.gpu_options.allow_growth = True
session = InteractiveSession(config=config)
height = 224
width = 224
nh=224
nw=224
ncol=3
# загружаем и создаем стандартную уже обученную сеть keras
visible2 = Input(shape=(nh,nw,ncol),name = 'imginp')
resnet = MobileNetV2(include_top=True,
weights="imagenet", input_tensor=visible2, input_shape=None, pooling=None,
classes=1000)
def read_image_files(files_max_count,dir_name):
    files = [item.name for item in os.scandir(dir_name) if item.is_file()]
    files_count = files_max_count
    if(files_max_count>len(files)): # определяем количество файлов не больше max
        files_count = len(files)
    image_box = [[]]*files_count
    for file_i in range(files_count): # читаем изображения в список
        image_box[file_i] = Image.open(dir_name+'/'+files[file_i]) # / ??
```

```
    return files_count, image_box
# возвращаем результаты работы нейронной сети
def getresult(image_box):
    files_count = len(image_box)
    images_resized = [[]]*files_count
# нормализуем изображения и преобразуем в numpy
    for i in range(files_count):
        images_resized[i] = np.array(image_box[i].resize((height,width)))/255.0
    images_resized = np.array(images_resized)
# подаем на вход сети изображение в виде numpy массивов
    out_net = resnet.predict(images_resized)
# декодируем ответ сети в один распознанный класс top=1 (можно больше классов)
    decode = decode_predictions(out_net, top=1)
    return decode
```

Содержимое файла st.sh

```
gunicorn --bind 127.0.0.1:5000 wsgi:app & APP_PID=$!
sleep 25 echo start
client python3
client.py APP_CODE=$?
sleep 5 echo $APP_PID
echo app code $APP_CODE
exit $APP_CODE
```

Содержимое файла wsgi.py

```
from some_app import app
if __name__ == "__main__":
    app.run()
```

Содержимое файла Net.html

```
{% extends "bootstrap/base.html" %}
{% import "bootstrap/wtf.html" as wtf %}
{% block title %}This is an page{% endblock %}
{% block content %}
{{ wtf.quick_form(form, method='post',enctype="multipart/form- data",
action="net") }}
<h2>Classes: </h2> 84
<!-- проверяем есть ли данные классификации -->
{% if neurodic %}
```

```
<!-- запускаем цикл прохода по словарю и отображаем ключ-значение -->
<!-- классифицированных файлов -->
{% for key, value in neurodic.items() %}
<h3>{{key}}: {{value}}</h3>
{% endfor %}
{% else %}
<h3> There is no classes </h3>
{% endif %}
<h2>Image is here: </h2>
<!-- отображаем загруженное изображение с закругленными углами -->
{% if image_name %}
<p>{{image_name}}
<p><img src={{image_name}} class="imgrounded" alt="My Image" width = 224 height=224
/>
{% else %}
<p> There is no image yet </p>
{% endif %}
{% endblock %}
```

Содержимое файла simple.html

```
<html>
    <head>
    {% if some_str %}
        <title>{{ some_str }} </title>
    {% else %}
        <title>Hm, there is no string!</title>
    {% endif %}
    </head>
    <body>
        <h1 style= " color: {{ some_pars.color }};"> Hello, {{ some_pars.user
}}!</h1>
        <h2>{{ some_str }}!</h2>
    </body>
</html>
```

Содержимое файла file.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="file.xslt" ?>
<people>
<man id= "1">
<name>John</name>
<age>30</age>
<work>Driver</work>
</man>
<man id = "2">
<name>Lisa</name>
<age>20</age>
<work>Programmist</work>
</man>
</people>
```

1.3 Результат работы

Рис. 1.3 – обрабатываемое изображение

Hello World!

Рис. 1.4 – первая URL

Hello, Ivan!

Hello my dear friends!!

Рис. 1.5 – Data_to

{% extends "bootstrap/base.html" %} {% import "bootstrap/wtf.html" as wtf %} {% block title %}This is an page{% endblock %} {% block content %} { { wtf.quick_form(form, method='post',enctype="multipart/form- data", action="net") } }

Classes:

84 {% if neurodic %} {% for key, value in neurodic.items() %}

{{key}}: {{value}}

{% endfor %} {% else %}

There is no classes

{% endif %}

Image is here:

{% if image_name %}

{ {image_name} }

My Image {% else %}

There is no image yet

{% endif %} {% endblock %}

Рис. 1.6 – Веб сервис по обработке

87 87

1	John	30	Driver
2	Lisa	20	Programmist

Рис. 1.7- 1.8 – Произошла проблема с загрузкой некоторых библиотек.

Из-за этого программа работает не совсем корректно

2 Задание по варианту

Задание: Веб-приложение должно обеспечивать преобразование подаваемой на вход картинки путем устранения шума в соответствии с переданным параметром сглаживания или любого другого параметра (метод фильтрации можно выбрать самостоятельно). Должна быть возможность вывода графика распределения цветов на картинке и графика распределения шума.

Содержимое файла start.py:

```
print("Hello world") from flask
import Flask from matplotlib
import Flask from cv2
import axes as ax app = Flask(__name__)
@app.route("/") def hello():
    return " <html><head></head> <body> Hello World! </body></html>"
    from flask import render_template
from flask_wtf import FlaskForm,RecaptchaField
from wtforms import StringField, SubmitField, TextAreaField
from wtforms.validators import DataRequired
from flask_wtf.file import FileField, FileAllowed, FileRequired
SECRET_KEY = 'secret'
app.config['SECRET_KEY'] = SECRET_KEY app.config['RECAPTCHA_USE_SSL'] = False
app.config['RECAPTCHA_PUBLIC_KEY'] = '6LcWcTsgAAAAAPB8sVkHoCz-15WrSN_pJCQYsQ_J'
app.config['RECAPTCHA_PRIVATE_KEY'] = '6LcWcTsgAAAAAMk9wxK1vCnJ6QiSSRYcUxFNhO2S'
app.config['RECAPTCHA_OPTIONS'] = {'theme': 'white'} from flask_bootstrap import
Bootstrap bootstrap = Bootstrap(app) class NetForm(FlaskForm):     size =
StringField('size', validators=[DataRequired()])     rcolor = StringField('choose
level of red intensity (0.0 - 1)', validators=[DataRequired()])
    gcolor = StringField('choose level of green intensity (0.0 - 1)',
validators=[DataRequired()])
    bcolor = StringField('choose level of blue intensity (0.0 - 1)',
validators=[DataRequired()])     upload = FileField('Load image',
validators=[FileRequired(),FileAllowed(['jpg', 'png', 'jpeg'], 'Images only!')])
recaptcha = RecaptchaField()     submit = SubmitField('send') from werkzeug.utils
import secure_filename import os import numpy as np from PIL import Image import
matplotlib.pyplot as plt import seaborn as sns
 def
img = cv2.imread('temp1.jpg')
factor = int(input('factor:'))
dst = cv2.fastNlMeansDenoisingColored(img, None, factor, 10, 7, 21)

plt.subplot(121), plt.imshow(img)
plt.subplot(122), plt.imshow(dst)
plt.show()

color = ('b','g','r')
for i,col in enumerate(color):
    histr = cv2.calcHist([img],[i],None,[256],[0,256])
    plt.plot(histr,color = col)
    plt.xlim([0,256])
plt.show()
```

```
@app.route("/net", methods=['GET', 'POST']) def net():
    form = NetForm()
    filename = None
    newfilename = None
    grname = None
    if form.validate_on_submit():         filename = os.path.join('./static',

secure_filename(form.upload.data.filename))
    sz = form.size.data
    sr = form.rcolor.data
    sg = form.gcolor.data
    sb = form.bcolor.data
    form.upload.data.save(filename)


newfilename, grname = draw(filename, sz, sr, sg, sb)

return render_template('net.html', form=form, image_name=newfilename, gr_name=grname)
if __name__ == " __main__ ":
    app.run(debug=True, host='127.0.0.1', port=5000)
```

Содержимое файла net.html:

```
{% extends "bootstrap/base.html" %}
{% import "bootstrap/wtf.html" as wtf %}
<!-- задаем заголовок страницы -->
{% block title %}This is an page{% endblock %}
<!-- блок body -->
{% block content %}
{{ wtf.quick_form(form, method='post',enctype="multipart/form-data", action="net") }}
        <table border="1">
    <tr>
      <td>Исходная1</td>
      <td>Новое</td>
      <td>график</td>
    </tr>
    <tr>


      <td>
        {% if ishd1 %}
        <p>{{ishd}}
        <p><img src={{ishd1}}/>
        {% else %}
        <p> There is no image yet </p>
        {% endif %}
    </td>


    <td>
```

```
        {% if image1_name %}
        <p>{{image1_name}}
        <p><img src={{image1_name}}/>
        {% else %}
        <p> There is no image yet </p>
        {% endif %}
    </td>
     <td>
        {% if gr_name %}
        <p>{{gr_name}}
        <p><img src={{gr_name}}/>
        {% else %}
        <p> There is no image yet </p>
        {% endif %}
    </td>

    </tr>

  </table>

{% endblock %}
```

Результат работы приложения:

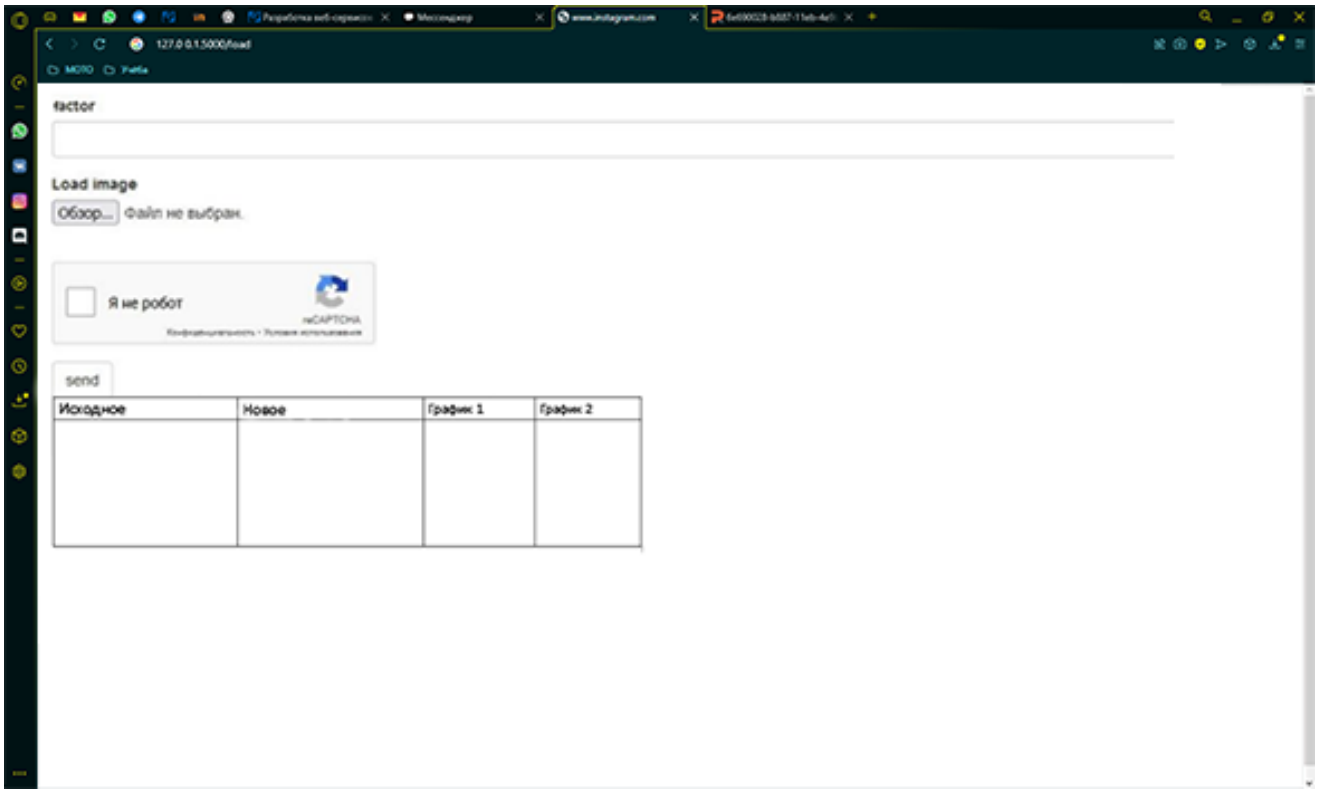

Рис 2.1–результат работы приложения

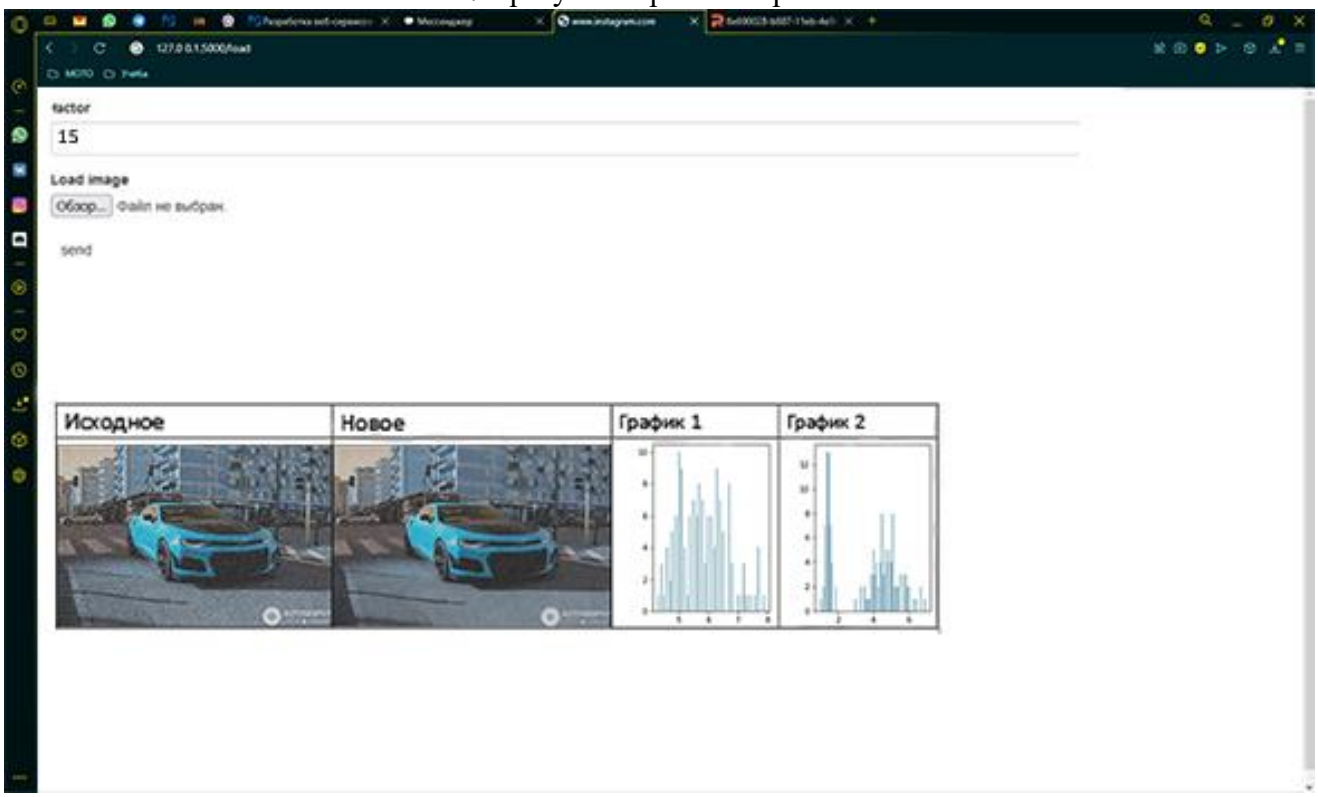

Рис 2.2–результат работы приложения

Вывод

Я изучил возможности языка Python, возможности по обработке данных, создал веб-приложение с использованием Flask.